# 第三章 栈和队列的结构分析与应用 · 教案

> 课程：数据结构（C 语言版）　教材：李刚等《数据结构（C语言版）》微课第 2 版 · 人邮
> 章节：第 3 章 栈和队列的结构分析与应用（教材 P38 ~ P57）
> 课时：4 课时（2 次课，每次 2 课时）　授课对象：计算机类高职 / 应用型本科一年级
> 配套：《第三章 栈和队列的结构分析与应用-C语言知识准备》《第三章 栈和队列的结构分析与应用-同步练习》

---

## 一、教材与学情分析

**教材分析：** 本章是线性表的"限制版"——栈和队列仍是线性结构，只是把插入、删除的位置**限制在端点。**因为限制，算法反而比顺序表、单链表更简单；难点从"搬数据"转移到了**边界条件**（`top` 的 `-1` 与 `StackSize-1`）和**环形取模**（`(rear+1)%N`）。本章还首次出现"用数据结构解决实际问题"的完整案例（括号匹配、进制转换、表达式求值），是从"会写代码"走向"会用结构"的转折点。

**学情分析：**

| 已有基础 | 可能存在的困难 |
| --- | --- |
| 会定义结构体、用数组、写 `for` 循环 | 分不清 `top` 是"下标"还是"指针"，空栈误写成 `top == 0` |
| 学过顺序表、单链表，会 `malloc` / `free` | 循环队列的取模运算不熟，队满条件写错 |
| 会写 `InitList(&L)` 这类传址函数 | 运算符优先级混淆：`*S->top` 与 `(*S)->top` |
| 对"后进先出"有生活直觉（弹夹、餐盘） | 一说"栈"就想到"栈内存"，与数据结构概念混淆 |

**对策：** 全程用**实物类比 + 手绘图 + 单步演示**（纸上画 `top` 移动），代码只写"最小可运行版本"，把边界条件单独抽出来做对比练习。

---

## 二、教学目标

### 【知识目标】

1. 掌握栈的定义及基本运算（`InitStack`、`StackEmpty`、`StackFull`、`Push`、`Pop`、`StackTop`）。
2. 掌握顺序栈和链栈基本运算的实现算法。
3. 掌握队列的定义及基本运算（`InitQueue`、`QueueEmpty`、`QueueFull`、`EnQueue`、`DeQueue`、`QueueFront`）。
4. 掌握循环队列和链队列基本运算的实现算法。

### 【能力目标】

1. 具备恰当地选择栈或队列作为数据的**逻辑结构**、选择顺序栈 / 顺序队列 / 链栈 / 链队列作为**存储结构**的能力。
2. 具备用 C 语言编程实现栈和队列基本运算的能力。
3. 具备运用栈和队列的知识解决实际问题的能力（括号匹配、进制转换、回文判断、表达式求值）。

### 【素养目标】

1. 具备专注、细致、严谨、精益求精的工匠精神（边界条件差 1 就全错）。
2. 具备敬业、踏实肯干的精神，弘扬劳动精神（从"食堂排队""弹夹装弹"看结构与生活）。

---

## 三、教学重点与难点

| | 内容 | 突破方法 |
|---|---|---|
| **重点** | 顺序栈的入栈 / 出栈；循环队列的入队 / 出队 | 手绘 `top`、`front`、`rear` 移动图，学生跟着画一遍 |
| **重点** | 栈"后进先出"、队列"先进先出"的判定与应用场景选择 | 生活实例对比 + 判断题快速抢答 |
| **难点** | 循环队列**队空 / 队满**的判断与"牺牲一个单元"的原因 | 画 4 格环形图，演示 `front == rear` 的两种含义，让学生自己发现冲突 |
| **难点** | 取模运算 `%` 在环形队列中的作用 | 先用钟表演示 `(11+2)%12`，再迁移到数组下标 |
| **难点** | 链栈 / 链队列的指针操作（先接后、再断前） | 复用第二章"先接后、再断前"口诀，只在表头 / 表尾各走一遍 |

---

## 四、教学方法与手段

- **类比教学法**：弹夹（栈）、食堂排队（队列）、钟表（取模）。
- **任务驱动法**：每课时给一个"能跑起来"的最小任务（如"让 3 个元素进栈再出栈"）。
- **对比教学法**：顺序栈 ↔ 顺序表、链栈 ↔ 单链表、循环队列 ↔ 钟表，用表格横向对比。
- **错误示范法**：故意写出 `top == 0` 判空、`rear++` 不取模的错误代码，让学生找错。
- 手段：PPT 动画（`top` 指针移动）、板书手绘、课堂在线抢答、课中 5 分钟上机。

---

## 五、课时安排

| 课时 | 内容 | 对应教材 | 关键产出 |
|---|---|---|---|
| 第 1 课时 | 栈的定义与基本运算、顺序栈的实现 | 3.1.1 ~ 3.1.2（P38-40） | 能默写顺序栈 6 个函数 |
| 第 2 课时 | 链栈、栈的应用（括号匹配、进制转换） | 3.1.3 及案例 | 能独立完成括号匹配 |
| 第 3 课时 | 队列的定义与基本运算、循环队列 | 3.2.1 ~ 3.2.2（P43-45） | 能写出队空 / 队满 / 个数公式 |

| 课时 | 内容 | 对应教材 | 关键产出 |
|---|---|---|---|
| 第 4 课时 | 链队列、栈与队列对比、习题讲评 | 本章小结 + 同步训练（P56-57） | 能完成同步训练并说明理由 |

# 六、教学过程设计

## 第 1 课时：栈的定义与顺序栈

| 环节 | 时间 | 教师活动 | 学生活动 | 设计意图 |
|---|---|---|---|---|
| 导入 | 5' | 展示手枪弹夹 / 食堂餐盘 GIF：最后压进去的子弹最先射出。提问："生活里还有哪些'后进先出'？" | 举例：撤销键、浏览器后退、叠盘子 | 用直觉建立"后进先出"概念，降低抽象度 |
| 概念建构 | 10' | 板书栈的定义：**只允许在一端进行插入和删除的线性表。**标出栈顶、栈底；给出 `a1…an` 的进栈顺序图 | 在笔记本上画出栈顶 / 栈底 | 明确"受限"在哪里，为后续"为什么简单"埋伏笔 |
| 基本运算 | 8' | 逐个讲解 6 个运算的**语义**（先不写代码）：初始化、判空、判满、进栈、出栈、取栈顶 | 口述每个运算的作用 | 先懂"做什么"，再学"怎么写" |
| 顺序栈 | 15' | ① 画空栈 `top=-1`；② 演示入栈 `top` 从 -1→0→1→2；③ 强调"**入栈先挪再放，出栈先取再挪**"；④ 板书 `SeqStack` 定义与 `Push` / `Pop` | 跟着在纸上画 `top` 移动，两人一组互相检查 | 突破重点；`top` 的移动是全部算法的基础 |
| 边界强化 | 5' | 出 3 道判断题：`top==0` 是空栈吗？`top==StackSize` 是满吗？`StackSize=5, top=2` 有几个元素？ | 抢答并说明理由 | 用错误选项制造认知冲突 |
| 小结 / 作业 | 2' | 归纳"一个变量管到底"；布置：默写 6 个函数 + 教材 P39 图示填空 | 记录 | 固化记忆 |

## 第 2 课时：链栈与栈的应用

| 环节 | 时间 | 教师活动 | 学生活动 | 设计意图 |
|---|---|---|---|---|
| 复习 | 5' | 随机抽 2 人上黑板默写 `Push` / `Pop` | 板演，其余同学找错 | 检测上节课吸收情况 |
| 链栈引入 | 8' | 提问："数组开小了会溢出，开大了会浪费，怎么办？"引出链栈；指出**链栈就是只在表头操作的单链表** | 回答：用链表按需申请 | 把新知识挂到旧知识上（第二章头插法） |

| 环节 | 时间 | 教师活动 | 学生活动 | 设计意图 |
|---|---|---|---|---|
| 链栈实现 | 12' | 板演 `Push`（先接后、再断前）、`Pop`（备份→摘链→`free`）；追问："为什么这里用 `LinkStack *S` 二级指针？" | 与 `InitList(&L)` 类比作答 | 复用口诀，降低 `malloc` 恐惧 |
| 应用一：括号匹配 | 12' | 现场演示 `( [ ] )` 的栈变化（贴纸模拟），引出"最近一个 = 栈顶"；给出单种括号版本代码 | 判断 `(()`、`)(`、`([])` 是否匹配 | 让学生体会"要逆序 / 要就近配对就用栈" |
| 应用二：进制转换 | 5' | 演示 13 → 1101：除 2 取余，余数**倒着**取；说明"倒着"就是栈 | 口算 10 → 1010 | 同一结构、不同问题，强化迁移 |
| 小结 / 作业 | 3' | 归纳"栈的两个关键词：逆序、就近"；作业：同步练习应用题 1、2 | 记录 | 为下节课队列做过渡 |

## 第 3 课时：队列与循环队列

| 环节 | 时间 | 教师活动 | 学生活动 | 设计意图 |
|---|---|---|---|---|
| 导入 | 5' | 食堂排队图：先来的先打饭。提问："如果队伍前面的人打完走了，后面的人要不要整体往前挪？" | 回答：不用挪，队伍"空一格"就行 | 引出"假溢出"问题，制造需求 |
| 队列概念 | 8' | 定义：只允许一端插入、另一端删除；标出队头 / 队尾；讲 6 个基本运算 | 画队列示意，标 `front` / `rear` | 与栈形成对照（一端 vs 两端） |
| 假溢出 | 7' | 用 5 格数组演示：连续入队出队后 `rear` 到末尾，但前面有空位 → 无法再入队，这就是**假溢出** | 观察并说出"前面明明还有空位" | 让"循环"成为学生自己想出来的方案，而不是被告知 |
| 循环队列 | 15' | ① 画环形图；② 讲 `(rear+1)%N` 绕回；③ **重点**：`front==rear` 既空又满 → 牺牲一个单元；④ 板书队空 / 队满 / 个数三个公式 | 跟着算：`N=4, front=3, rear=1` 时元素个数 | 突破本章最大难点，用"环形跑道"形象化 |
| 单步演练 | 7' | 按教材 P44 的 4 幅图，逐步演示"空 → A,B 入队 → C 入队队满 → 全部出队队空" | 每步先猜 `front`、`rear` 的值再看答案 | 把公式变成"看得见的走位" |
| 小结 / 作业 | 3' | 三句话总结：绕圈用取模、空是同点、满是差一格 | 复述 | 便于记忆 |

### 第 4 课时：链队列、对比总结与习题讲评

| 环节 | 时间 | 教师活动 | 学生活动 | 设计意图 |
|---|---|---|---|---|
| 复习检测 | 6' | 3 道速答题：队满条件？元素个数公式？为什么少用一个单元？ | 抢答 | 检测难点是否真正掌握 |
| 链队列 | 12' | 讲"带头结点的单链表 + 队头指针 + 队尾指针"；入队在**表尾**（尾插法），出队在**表头**（删首结点）；强调出队后若队列变空，`rear` 要回到头结点 | 与链栈对比：一个在头、一个在尾 | 一次讲清"链栈改头、链队改尾" |
| 四结构对比 | 10' | 板书对比表：顺序栈 / 链栈 / 循环队列 / 链队列（判空、判满、时间、空间、适用场景） | 补全表格中的空格 | 形成结构化认知，应对选择题 |
| 选型讨论 | 5' | 给 4 个场景（浏览器后退、打印任务、括号检查、操作系统进程调度），让学生选结构并说理由 | 小组讨论 1 分钟，代表发言 | 落实"能力目标 1：会选择" |
| 习题讲评 | 10' | 讲同步训练中的易错题：`SSXSXSSXXX` 出栈序列、`fun` 程序段功能分析 | 先独立做，再对照讲解 | 从"懂"到"会做题" |
| 课堂小结 | 2' | 一句话收束：**栈是一端进出的"弹夹"，队列是两端分工的"队伍"；一个盯 `top`，一个盯 `front`/`rear` 绕圈** | 记录 | 形成整体印象 |

## 七、板书设计

```
                    第三章　栈和队列

┌──────────────────────────────┬──────────────────────────────┐
│  栈 Stack（LIFO）            │  队列 Queue（FIFO）          │
│  只在一端增删                │  一端进、另一端出            │
│  栈顶 top ↑   栈底           │  队头 front ← → 队尾 rear    │
├──────────────────────────────┼──────────────────────────────┤
│  顺序栈                      │  循环队列                    │
│  top=-1        空            │  front==rear        空       │
│  top=N-1       满            │  (rear+1)%N==front  满       │
│  入：++top 再放              │  入：放 data[rear]，rear++   │
│  出：先取 再 top--           │  出：取 data[front]，front++ │
│  个数 = top+1                │  个数 =(rear-front+N)%N      │
├──────────────────────────────┼──────────────────────────────┤
│  链栈 = 只在表头操作         │  链队 = 表尾入、表头出       │
│  入栈：s->next=*S; *S=s;     │  出队后若空，rear 回头结点   │
└──────────────────────────────┴──────────────────────────────┘
应用：括号匹配、进制转换、回文、表达式求值（要逆序/就近 → 栈）
```

# 八、作业与评价

**作业安排：**

1. **基础题（全体）**：默写顺序栈 `Push` / `Pop` 与循环队列 `EnQueue` / `DeQueue`，并标注每步含义。
2. **提高题（全体）**：教材 P56 同步训练一、二（填空、选择）。
3. **应用题（选做）**：P57 应用题 2、3，写出程序段功能。
4. **算法设计（学有余力）**：用栈判断回文；求循环链表中结点个数。

**评价方式：**

| 维度 | 权重 | 依据 |
| --- | --- | --- |
| 课堂参与（抢答、板演、找错） | 20% | 课堂记录 |
| 代码实现（顺序栈 + 循环队列可运行） | 40% | 上机验收 |
| 同步练习正确率 | 25% | 批改 |
| 应用分析（选结构、说理由） | 15% | 小组发言 |

---

# 九、教学反思要点

1. `top == -1` **与** `top == 0` **的混淆**是否通过手绘图解决？若仍有学生错，下节课用"索引卡片"现场摆位再演一遍。
2. **队满条件**是本章最大失分点，需检查学生是否能解释"为什么牺牲一个单元"，而不只是背公式。
3. 链栈 / 链队列的指针顺序是否还有"先断后接"的错误？可增加 30 秒"口诀接龙"（先接后、再断前）。
4. 应用案例（括号匹配）是否只停留在"看懂"？下次可改为**先让学生用自然语言写出匹配规则**，再翻译成代码，减少直接给代码的比例。
5. 关注基础薄弱学生：可让其只完成"顺序栈 + 循环队列"的填空版代码，先获得成功体验，再补链式存储。